EduManage

Starting the application

Prerequisite: Node.js installed (needed to run the launcher and the frontend).

Run one of the following from the project root:

```
node scripts/dev.js
```

macOS/Linux:

```
./start.sh
```

Windows:

```
start.bat
```

This installs any missing dependencies (including Python via `uv` for the backend), applies database migrations, seeds baseline data, and starts both servers:

- Backend: http://127.0.0.1:8000
- Frontend: http://127.0.0.1:3000

Press `Ctrl+C` to stop both servers.

Scope: the 7 objectives

Every screen and API in this codebase exists to serve one of these seven objectives — there is no unrelated functionality in scope. Each maps to a specific implementation phase (see `docs/tasks.md` for the full build log):

#	Objective	Implemented as
1	Fee & Financial Management	Phase 2 — categories, structures, invoices, payments, discounts, credits, financial reports

#	Objective	Implemented as
2	Attendance Management	Phase 3 — daily marking, lock windows, absenteeism detection, excuse requests, reports
3	Communication & Notifications	Phase 5 — in-app/email notifications, announcements, calendar, preferences
4	Student Information Management	Phase 1 — registration, guardians, documents, academic history, class allocation
5	Academic Performance	Phase 4 — assessments, scores, weighted averages, at-risk detection
6	Examination Management	Phase 4 — exams, schedules, results, report cards, class ranking
7	Staff & Teacher Management	Phase 1 — staff records, class/section assignments, staff attendance, documents

Anything not on this list (a live payment gateway, a native mobile app, attendance/performance correlation analytics) is intentionally **not built** — tracked only as "fast-follow" ideas in `docs/tasks.md` and not reachable from the app.

Current verified state (2026-08-30): backend test suite 98/98 passing, frontend production build clean across all 59 routes, `eslint` clean. `docs/tasks.md`'s Phase 4 frontend checklist had gone stale (marked several screens as "not built" that were actually completed in a later pass); it's been corrected to match the working tree.

Testing guide: verifying each objective

0. Prerequisites — log in

1. Start the app (see above).
2. The seeded Admin login comes from `backend/.env` (`ADMIN_EMAIL` / `ADMIN_PASSWORD` — see `backend/.env.example` for the format; the seed script refuses to run with the placeholder default, so a real value must already be set for your local `.env`). Sign in at http://127.0.0.1:3000/login.
3. Only the Admin account is seeded by default — Principal/Registrar/Accountant/Teacher/Student/Parent all need to be created (see each objective below, and the Parent/Student appendix at the end for the one role that needs an API workaround).

Every objective below assumes you're starting from a fresh seed (one Admin, one demo academic year/term/class/section, no students/staff yet) unless noted.

1. Fee & Financial Management

Covers: fee categories/structures, invoice generation, payments with auto-allocation, discounts, credits, receipts, financial reports.

1. As Admin, go to **Fees → Categories** (`/fees/categories`) — confirm the seeded categories (Tuition, Development Levy, PTA/SDC, Sports, ICT, Exam Fee) are listed and editable.
2. Go to **Fees → Structures** (`/fees/structures`), create a fee structure for the current term/class, then use its **Generate invoices** action.
3. Open a student's profile → **Fees** tab: confirm the new invoice appears in the ledger and term summary.
4. Use **Record Payment** on the student's Fees tab for an amount less than the invoice total — confirm the invoice shows `partial` status and the correct outstanding balance.
5. Record a second payment that overpays — confirm the oldest-outstanding-invoice-first allocation runs, and any remainder becomes an available credit (Credits panel on the same tab).
6. Go to **Fees → Discounts** (`/fees/discounts`), submit a discount above the approval threshold (`system_settings.fee_discount_approval_threshold_cents`) and confirm it lands in the pending-approval queue; approve it as Admin/Principal.
7. Go to **Fees → Reports** (`/fees/reports`) — confirm collection rate, credit liability, discount utilization, and cash-up (filterable by date) all render with the data from steps above. **Outstanding Balances** has its own page, linked from here.
8. Download a receipt PDF from a recorded payment.

2. Attendance Management

Covers: daily marking, edit lock windows, absenteeism detection, excuse requests, reports.

1. Log in as a Teacher assigned to a section (create one via Objective 7 first if none exists).
2. Go to **Teacher → Attendance** (`/teacher/attendance`), mark the roster present/absent/late for today, and save.
3. Try editing a record after the lock window (`system_settings.attendance_edit_lock_hours`) — confirm it's rejected for a Teacher, then confirm an Admin can override it (and that the override is audited).
4. Mark the same student absent on 3 consecutive days — confirm they appear on **Attendance → Absenteeism Watchlist** (`/attendance/watchlist`), which reads `absenteeism_consecutive_absences_trigger` .

5. As a Parent/Student (or via the student's Attendance tab on the admin profile), submit/approve an excuse request from **Teacher → Excuse Requests** (`/teacher/excuse-requests`).
6. Check **Attendance → Section Report** (`/attendance/reports/section`) reflects the marks entered.
7. Confirm the Student/Parent calendar view of the same attendance data matches what was marked (same shared component is used in all three surfaces, so numbers should never disagree).

3. Communication & Notifications

Covers: in-app + email notifications, announcements, school calendar, notification preferences.

1. As Admin/Principal, go to **Announcements** (`/announcements`), compose a school-wide announcement (try both a normal and a `safety` -flagged one — safety is Admin/Principal-only).
2. As a Teacher, confirm the composer is locked to their own current-term section (no safety option) and that a scoped announcement is only visible to that section's guardians/students, not the whole school.
3. Confirm the notification bell (top-right, every role) shows an unread badge and the new announcement in its dropdown; click "View all" → **Notification Center**, and "Preferences" → per-category settings.
4. Trigger a real notification indirectly: generate a fee invoice (Objective 1) or mark 3 consecutive absences (Objective 2) — confirm the guardian/student receives an in-app notification without any manual send.
5. In **Settings → Notifications** (Admin), try disabling a mandatory category (`fees` or `safety`) — confirm it's blocked (mandatory categories can't be disabled).
6. Go to **Calendar** (`/calendar`), create an event as Admin/Principal/Teacher (any `events:manage` holder), and confirm Parent/Student see it read-only on their own calendar.

Known gap: fee due-date/overdue reminders and academic at-risk alerts are not wired yet (no scheduler infrastructure exists) — don't expect those specific triggers to fire.

4. Student Information Management

Covers: registration, guardians, documents, academic history, section allocation/transfer, withdrawal.

1. As Admin/Registrar, go to **Students → New** (`/students/new`), complete the registration wizard — confirm the admission number is generated (`ADM-<year>-<seq>`).
2. Open the new student's profile (`/students/{id}`) — walk each tab: **Overview**, **Guardians** (add one — note the duplicate-guardian check by phone/email), **Documents** (upload one;

confirm type/size validation), **Academic History**, **Fees**, **Attendance**.

3. Use **class-section allocation/transfer** from the student profile — confirm it writes to academic history and respects the section capacity check (try exceeding capacity to see the override+audit path).
4. Use the **Withdraw** action — confirm the confirmation dialog states the real consequence before proceeding.
5. Log in as the section's assigned Teacher and confirm the read-only roster shows the new student.

Known gap: there's no simplified read-only Parent/Student "my profile" screen yet — Parent/Student access their own data through the Fees/Attendance/Performance/Report-card tabs individually, not one consolidated profile page.

5. Academic Performance

Covers: assessment types, assessments, bulk score entry, weighted averages, at-risk detection.

1. As Admin, go to **Academics → Assessment Types** (`/academics/assessment-types`) and create at least one (e.g. "CALA", "Test") — none are seeded by default, so assessments can't be created until this exists.
2. As the section's Teacher, go to **Teacher → Assessments** (`/teacher/assessments`), create an assessment with a weight, then go to **Teacher → Gradebook** (`/teacher/gradebook`) and bulk-enter scores for the roster (including marking one student absent).
3. Confirm the weighted term average appears correctly once enough assessments are scored.
4. Check **Teacher → Performance** (`/teacher/performance`) and **Academics → Performance Reports** (`/academics/performance-reports` , Admin) — confirm the at-risk watchlist reflects `system_settings.academic_at_risk_threshold_pct` .
5. As Student/Parent, confirm their own performance page (`/student/performance` or `/parent/performance`) shows the entered scores immediately — coursework has **no publish gate**, unlike exams below.

6. Examination Management

Covers: exam scheduling, mark entry, publish gating, report cards, class ranking.

1. As Admin, go to **Exams** (`/exams`), create an exam and schedule it for a subject/section.
2. As the Teacher, go to **Teacher → Exams** (`/teacher/exams`) and bulk-enter marks for the schedule.
3. Before publishing, confirm the Student/Parent view of that exam shows an empty result set (not an error) — this is deliberate query-time gating, not a 403.

4. Publish the exam (Admin/Principal or a `publish` permission holder) — confirm Student/Parent now see the result, and a notification fired.
5. Go to **Report Cards** (`/report-cards` Admin, or `/teacher/report-cards`), compile a report card for a student — try compiling with a missing subject's marks first to confirm it's blocked with a clear message naming the subject, then complete the marks and compile successfully.
6. Publish the report card, confirm the PDF download works, and that class rank appears (or is hidden) according to `system_settings.class_ranking_enabled` .

7. Staff & Teacher Management

Covers: staff records, onboarding, class/section assignment, staff attendance, documents. **This is also how you create a Teacher login to test Objectives 2, 5, and 6 above.**

1. As Admin, go to **Staff → New** (`/staff/new`), fill in the details with `role_codes` including `teacher` — this creates both the staff record and a linked login account via the invite flow.
2. Check the **backend terminal output** (the `uvicorn` process started by `scripts/dev.js`) for a line like `Staff invite created for <email> — set-password token: <token>` — no email delivery is wired in dev, so this is the only place the token appears.
3. Manually visit `http://127.0.0.1:3000/reset-password?token=<token>` to set that teacher's password (this screen is reused for the invite flow — no separate accept-invite screen exists yet).
4. Log in as the new Teacher account.
5. Back in Admin, go to **Staff → Assignments** (`/staff/assignments`), assign the new teacher to a section for the current term — confirm the one-teacher-per-class / one-class-per-teacher rule (try assigning a second teacher to the same section and confirm the `409 SECTION_ALREADY_ASSIGNED` message).
6. Open the staff profile (`/staff/{id}`) and check the **Overview/Assignment/Attendance/Documents** tabs; upload a staff document.
7. Log in as the Teacher and confirm **Teacher → My Class / My Profile** shows the assigned section correctly.

Known gap: there's no dedicated staff attendance *register* (mark/bulk-entry) screen yet — the staff profile's Attendance tab is read-only history only.

Appendix: testing the Parent/Student experience

There's currently no admin screen for granting a Parent or Student a portal login (`docs/tasks.md` tracks the general Admin users/roles screen as not yet built), so it needs one API call via Swagger UI at http://127.0.0.1:8000/docs (or `curl`), authenticated as Admin:

1. `POST /api/v1/users` with `{"email": "parent1@example.com", "role_codes": ["parent"]}` — note the returned `id`, then check the backend terminal for `Invite created for parent1@example.com — set-password token: <token>`.
2. Set the password via `http://127.0.0.1:3000/reset-password?token=<token>`, same as the staff flow above.
3. Link that login to a guardian: `POST /api/v1/guardians` with the guardian's details plus `"user_id": "<id from step 1>"`, then `POST /api/v1/students/{student_id}/guardians` to link that guardian to a student. (For a Student login instead of a Parent, create the user the same way, then `PATCH /api/v1/students/{student_id}` with `{"user_id": "<id>"}`.)
4. Log in as that email — you should land on the Parent/Student dashboard with that child's data scoped correctly.